

some notes

This note explains the mathematical objects used across `sensitivity_tools.py`, `experiment_common.py`, `direct_dynamics.py`, `mexican_hat_dynamics.py`, and the three experiment scripts (`asrnn_double_well_bifurcation_sensitivity.py`, `asrnn_henon_heiles_symmetry_sensitivity.py`, `asrnn_mexican_hat_symmetry_sensitivity.py`), and how to read the plots

1. Core objects

Given a trained network $f_\theta : \Omega \rightarrow \mathbb{R}^m$, define the **functional sensitivities**

$$S_i(x) = \frac{\partial f_\theta(x)}{\partial \theta_i}$$

and the **functional tangent space** $T_\theta = \text{span}\{S_i\} = \text{Im}(J_\theta)$, where $J_\theta = \partial f_\theta / \partial \theta$. T_θ is the set of function-space directions reachable by an infinitesimal parameter perturbation.

A continuous physical symmetry generator X (translation, rotation, scaling, or any other family direction) acts on a function via Xf . The **intrinsic symmetry space** is $\mathcal{G} = \text{span}\{X_1 f, \dots, X_k f\}$. The central question is how T_θ relates to $\mathcal{G}$:

- **Symmetry projection error:** $\varepsilon(X) = \|(I - \Pi_{T_\theta})Xf\|$.
- **Principal angles / representation dimension:** angles between T_θ and $\mathcal{G}$ when $\mathcal{G}$ has more than one generator; counting how many generators are represented below an angle threshold gives a **representation dimension** (0 to k).
- **Parameter attribution** (c_i , sec. 0.4): once Xf is (approximately) in T_θ , the minimum-norm coefficients solving $Xf \approx \sum_i c_i S_i$ identify *which* parameters realise the generator.
- **Sensitivity equivariance** (E_i , sec. 0.6): if $f_\theta(gx) = \rho(g)f_\theta(x)$ exactly for a group element g , then every sensitivity must transform the same way: $S_i(gx) = \rho(g)S_i(x)$. The per-parameter defect

$$E_i = \frac{\|S_i(gx) - \rho(g)S_i(x)\|}{\|S_i(x)\|}$$

measures how far parameter i 's sensitivity is from the equivariant behaviour the *function's own* symmetry would demand. Note that this is a different question from whether f_θ itself is symmetric – a network can compute a highly symmetric function while its individual parameter sensitivities are far from transforming correctly.

- **Energy conservation** Acts as another continuous symmetry (time translation) valid for every system and every architecture here except the MLP which needs to learn it.

2. `sensitivity_tools.py` function reference

Function	Computes	Notes
<code>tangent_projection(J, target, cutoff)</code>	$\varepsilon(X)$, principal angle to a single target, min-norm c_i , resolved rank	target is one generator direction, e.g. a physical family derivative or Xf evaluated on a probe grid. cutoff truncates J 's SVD so tiny numerical singular values.
<code>principal_angles_and_dimension(G, cutoff, angle_threshold)</code>	principal angles between resolved T_θ and resolved $\text{span}(G)$, and the representation dimension	G stacks multiple generator columns. Used to jointly test 2+ generators at once.

Function	Computes	Notes
<code>finite_transform_residual(v_x, v_gx, rep)</code>	relative residual of $f(gx) \approx \rho(g)f(x)$ for a finite group element g	<code>rep=None</code> means invariance (scalar potential); a matrix means equivariance (vector force). Works for any g (reflection, rotation by any angle) since it only needs values already evaluated at x and $g \cdot x$, Basically just implementing error in capturing discrete symmetries. self explanatory
<code>sensitivity_transform_residual(J_x, J_gx, rep)</code>	one aggregate relative Frobenius residual of $J(gx) \approx \rho(g)J(x)$	defined earlier
<code>per_parameter_equivariance_error(J_x, J_gx, rep)</code>	raw (unnormalised) $J(gx) - \rho(g)J(x)$	Building block for the two functions above.
<code>transform_defect(J_x, J_gx, rep)</code>	fraction of J 's energy that is an odd function of the check coordinate (kind useless but just in case)	self explanatory
<code>domain_parity_energy_fraction(J_x, J_flipped_x)</code>	pointwise dE_{true}/dt under the model's <i>learned</i> dynamics	Zero for the true dynamics, nonzero measures how much the learned flow fails to conserve the true physical energy.
<code>noether_energy_drift(grad_e_p, grad_e_q, dpdt, dqdt)</code>	RMS drift relative to the typical term magnitude	The scalar reported in every <code>energy_conservation.png</code> .
<code>relative_energy_drift(grad_e_p, grad_e_q, dpdt, dqdt)</code>	L2 norm of a raw per-parameter vector within each named layer	For magnitude-like quantities (sensitivities, attribution coefficients).
<code>aggregate_by_module(values, slices)</code>	mean within each named layer	For already-normalised ratios like E_i , where a norm would just grow with layer size.

3. Three architectures (`direct_dynamics.py`, `mexican_hat_dynamics.py`)

The double-well and Hénon-Heiles scripts support `--architecture hamiltonian` (default) or `--architecture direct_mlp`; the Mexican-hat script additionally supports `--architecture equivariant`:

- **hamiltonian**: the ASRNN repo's V_θ/K_θ split, integrated with a symplectic kick-drift-kick (Verlet) step. This architecture guarantees, by construction, that the symplectic integrator nearly conserves the network's *own* learned energy $H_\theta = V_\theta + K_\theta$
- **direct_mlp** (`direct_dynamics.DirectDynamicsMLP + DirectLeapfrogIntegrator`): one MLP mapping $(p, q, \alpha) \rightarrow (\dot{p}, \dot{q})$.
- **equivariant** (`mexican_hat_dynamics.EquivariantHamiltonianMLP`, Mexican-hat only): a V_θ/K_θ pair built from `escnn`'s SO(2)-equivariant layers, exactly rotation-invariant.

Output directories follow `outputs/<experiment>` (hamiltonian) and `outputs/<experiment>_direct_mlp / outputs/<experiment>_equivariant`. There is no V_{net} for `direct_mlp`, so potential-based diagnostics (potential invariance, double-well curvature) are skipped there (reported as NaN / omitted plots) – only force-level and sensitivity-level checks apply.

5. Double-well experiment (`asrnn_double_well_bifurcation_sensitivity.py`)

System: $H(p, q; \alpha) = p^2/2 + \alpha q^2/2 + q^4/4$, force $F_\theta(q, \alpha) = -\partial V_\theta/\partial q$ (or read directly from the model for `direct_mlp`). α crosses the pitchfork bifurcation at $\alpha = 0$.

Two generators are tested :

1. **Bifurcation direction** $\partial_\alpha F = -q$ (continuous, exact, model-independent): the direction the true force field moves in as you deform the physical family across the bifurcation. A genuine Xf in the PDF's sense.
2. **Z2 symmetry** $(q, p) \rightarrow (-q, -p)$ (discrete, exact for *any* α): V is even, F is odd. Discrete groups have no infinitesimal generator, so this is tested as a finite-transform residual (`finite_transform_residual` / `sensitivity_transform_residual`), not a projection.

Plots

- **bifurcation_sensitivity_summary.png**
 - *Curvature at $q=0$* : $\partial_q^2 V_\theta(0, \alpha)$ vs true curvature α (dashed). NaN for `direct_mlp` (no V_{net}).
 - *Representation of $\partial_\alpha F = -q$* : $\varepsilon(X)$ vs α .
 - *Tangent-space angle*: the corresponding principal angle, degrees.
 - *Total force sensitivity*: overall sensitivity magnitude.
- **parity_symmetry_summary.png**: potential/force parity residuals (NaN potential residual for `direct_mlp`), aggregate sensitivity equivariance, and odd-content fraction of the Jacobian.
- **equivariance_by_module.png** / **equivariance_scatter.png**: mean/ per-parameter E_i (sec 0.6), random init vs. trained, by layer. K_{net} is always exactly 0 (hamiltonian only; force doesn't depend on it). The scatter plot shows zero-initialised biases starting at exactly $E_i = 0$ and drifting *up* after training.
- **training_diagnostics.png**, **learned_potential_final.png** (skipped for `direct_mlp`) / **learned_force_final.png**.

CSVs: `top_parameters_step_*.csv` (RMS $|c_i|$), `top_equivariance_violations_step_*.csv` (RMS E_i).

6. Hénon-Heiles experiment (`asrnn_henon_heiles_symmetry_sensitivity.py`)

System: $V(q_1, q_2; \alpha_1, \alpha_2) = \frac{1}{2}(q_1^2 + q_2^2) + \alpha_1 q_1^2 q_2 - \frac{\alpha_2}{3} q_2^3$. When $\alpha_1 = \alpha_2$ this is the classical Hénon-Heiles potential with exact C_{3v} symmetry (120° rotation + reflections); training data only ever sits on this symmetric diagonal, so off-diagonal (α_1, α_2) probes are a genuine generalisation test.

Three generators are tested:

1. **Coupling-constant direction** $\partial_\alpha F = (-2q_1 q_2, q_2^2 - q_1^2)$ along the symmetric diagonal (continuous, exact, model-independent) – the direct analogue of the double well's bifurcation direction.
2. **120° (C3) rotation** (discrete, exact only when $\alpha_1 = \alpha_2$): tested as a finite-transform residual, evaluating the network directly at rotated probe points.
3. **q_1 -reflection** $(q_1, p_1) \rightarrow (-q_1, p_1 \rightarrow -p_1)$ (discrete, exact for **any** α_1, α_2).

Two analysis sweeps: **symmetric** ($\alpha_1 = \alpha_2 = \alpha$, several values) and **breaking** (fixed α_1, α_2 swept away from it), to see how each residual grows with $|\alpha_1 - \alpha_2|$.

Plots:

- **symmetric_diagonal_summary.png** (symmetric sweep, vs α): C3 force-equivariance residual, C3 sensitivity-equivariance residual (aggregate), and the coupling-generator $\varepsilon(X)$ /angle.
- **symmetry_breaking_summary.png** (breaking sweep, vs $\alpha_2 - \alpha_1$): C3 rotation residual (should grow with detuning) side-by-side with the q_1 -reflection residual (should **not** depend on detuning – it's exact everywhere; this panel is the control).
- **equivariance_by_module.png** / **rotation_equivariance_scatter.png** / **reflection_equivariance_scatter.png**: mean/per-parameter E_i for both generators, random init vs. trained, at the central symmetric α .
- **learned_force_field_final.png**: field plot of true vs learned force field.
- **training_diagnostics.png**.

CSVs: `top_coupling_parameters_*.csv` (RMS $|c_i|$), `top_equivariance_violations_*.csv` (RMS E_i for rotation).

7. Mexican-hat experiment (`asrnn_mexican_hat_symmetry_sensitivity.py`, new)

System: $V(q_1, q_2; \alpha) = \frac{1}{2}\alpha r^2 + \frac{1}{4}r^4$, $r^2 = q_1^2 + q_2^2$ – the direct 2D generalisation of the double well. Unlike Hénon-Heiles, V depends on q only through r^2 , so it is **exactly continuously rotation-invariant for every α** .

Two continuous generators:

1. **Bifurcation direction** $\partial_\alpha F = -(q_1, q_2)$
2. $X_{\text{rot}}F$: Rotational symmetry generator

mexican_hat_summary.png (6 panels, vs α , colour = checkpoint): finite-rotation force/sensitivity equivariance, bifurcation-generator projection error, $X_{\text{rot}}F$ projection error, joint representation dimension.

CSVs: `top_bifurcation_parameters_*.csv`, `top_xrot_parameters_*.csv` (RMS $|c_i|$ for each generator), `top_equivariance_violations_*.csv` (RMS E_i for rotation).

8. The equivariant architecture (`--architecture equivariant`, Mexican-hat only (cause $\text{SO}(2)$))

This section considers an equivariant architecture.

Construction (`mexican_hat_dynamics.EquivariantHamiltonianMLP`, `InvariantScalarMLP`): both $V_\theta(q_1, q_2, \alpha)$ and $K_\theta(p_1, p_2)$ are built as `escnn.nn.NormPool`.